

instanceof operator :

It is an operator as well as keyword in java.

* It is used to verify whether a reference variable is pointing to a particular type of object or not.

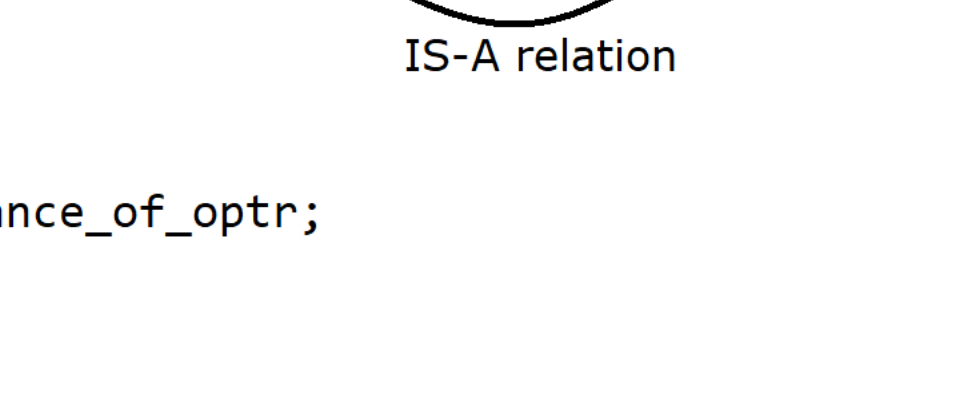
Example :

```
Test t1 = new Test();
boolean isInstance = t1 instanceof Test;
```

* It is a relational operator so, it will return true OR false.

* It is mainly used to resolve the issue of **java.lang.ClassCastException**

* While working with **instanceof operator**, In between reference variable and class/interface type we must have **IS-A relation otherwise we will get a compilation error.**



```
package com.ravi.instance_of_optr;
```

```
class Test
{
}
public class InstanceOfOperatorDemo1 {
    public static void main(String[] args)
    {
        Test t1 = new Test();
        boolean isInstance = t1 instanceof Test;
        IO.println(isInstance);
    }
}
```

```
package com.ravi.instance_of_optr;
```

```
class Test
{
}
public class InstanceOfOperatorDemo1 {
    public static void main(String[] args)
    {
        Test t1 = new Test();
        if(t1 instanceof Test)
        {
            IO.println("t1 is pointing to Test object");
        }
    }
}
```

```
package com.ravi.instance_of_optr;
```

```
class A
{
}
class B extends A
{
}
class C extends B
{
}
public class InstanceOfOpDemo2 {
    public static void main(String[] args)
    {
        C c1 = new C();
        if(c1 instanceof C)
        {
            IO.println("C1 is pointing to C class");
        }
        if(c1 instanceof B)
        {
            IO.println("C1 is pointing to B class");
        }
        if(c1 instanceof A)
        {
            IO.println("C1 is pointing to A class");
        }
        if(c1 instanceof Object)
        {
            IO.println("C1 is pointing to Object class");
        }
    }
}
```

```
package com.ravi.instance_of_optr;
```

```
class Bird
{
}
class Fish
{
}
public class InstanceOfOpDemo3
{
    public static void main(String[] args)
    {
        Fish f = new Fish();
        if(f instanceof Bird) //error Fish & Bird is not in IS-A relation
        {
        }
    }
}
```

```
package com.ravi.instance_of_optr;
```

```
class Vehicle
{
    public void run()
    {
        System.out.println("Vehicle is running");
    }
}
```

```
class BMW extends Vehicle
{
}
```

```
class Audi extends Vehicle
{
}
```

```
public class InstanceOfOpDemo4
{
    public static void main(String[] args)
    {
        String choice = IO.readLine("Enter your car choice BMW/Audi :");
        Vehicle v = null;
        if(choice.equalsIgnoreCase("BMW"))
        {
            v = new BMW();
            acceptCarType(v);
        }
        else
        {
            v = new Audi();
            acceptCarType(v);
        }
    }
    public static void acceptCarType(Vehicle v)
    {
        if(v instanceof BMW)
        {
            System.out.println("BMW Car");
        }
        else
        {
            System.out.println("AUDI Car");
        }
    }
}
```

Co-Variant return type :

As we know while working with method overriding, the method signature must be same as well as return type must be compatible,

If return type is **not compatible** we will get compilation error as shown in the program.

```
class A
{
    public void show()
    {
    }
}
class B extends A
{
    @Override
    public double show()
    {
        return 0.0;
    }
}
public class CompatibleReturnType
{
    public static void main(String[] args)
    {
        System.out.println("Hello World!");
    }
}
```

Note : error, return type double is not compatible with void

But from JDK 1.5V onwards we can change the return type of the method in only one case that the return type of both the METHODS(SUPER AND SUB CLASS METHODS) MUST BE IN INHERITANCE RELATIONSHIP (IS-A relationship so it is compatible) called Co-Variant as shown in the program below.

Note :- Co-variant will not work with primitive data type, it will work only with reference type.

***Co-variant represents only **one direction** that means sub class method return type object we can assign super class method return object i.e in one direction. (A a = new B(); The Object of B we can assign to A)

```
class Animal
{
}
class Dog extends Animal
{
}
class Shape
{
    public Animal draw()
    {
        System.out.println("Generic Draw");
        return null;
    }
}
class Square extends Shape
{
    @Override
    public Dog draw()
    {
        System.out.println("Drawing Square");
        return null;
    }
}
```

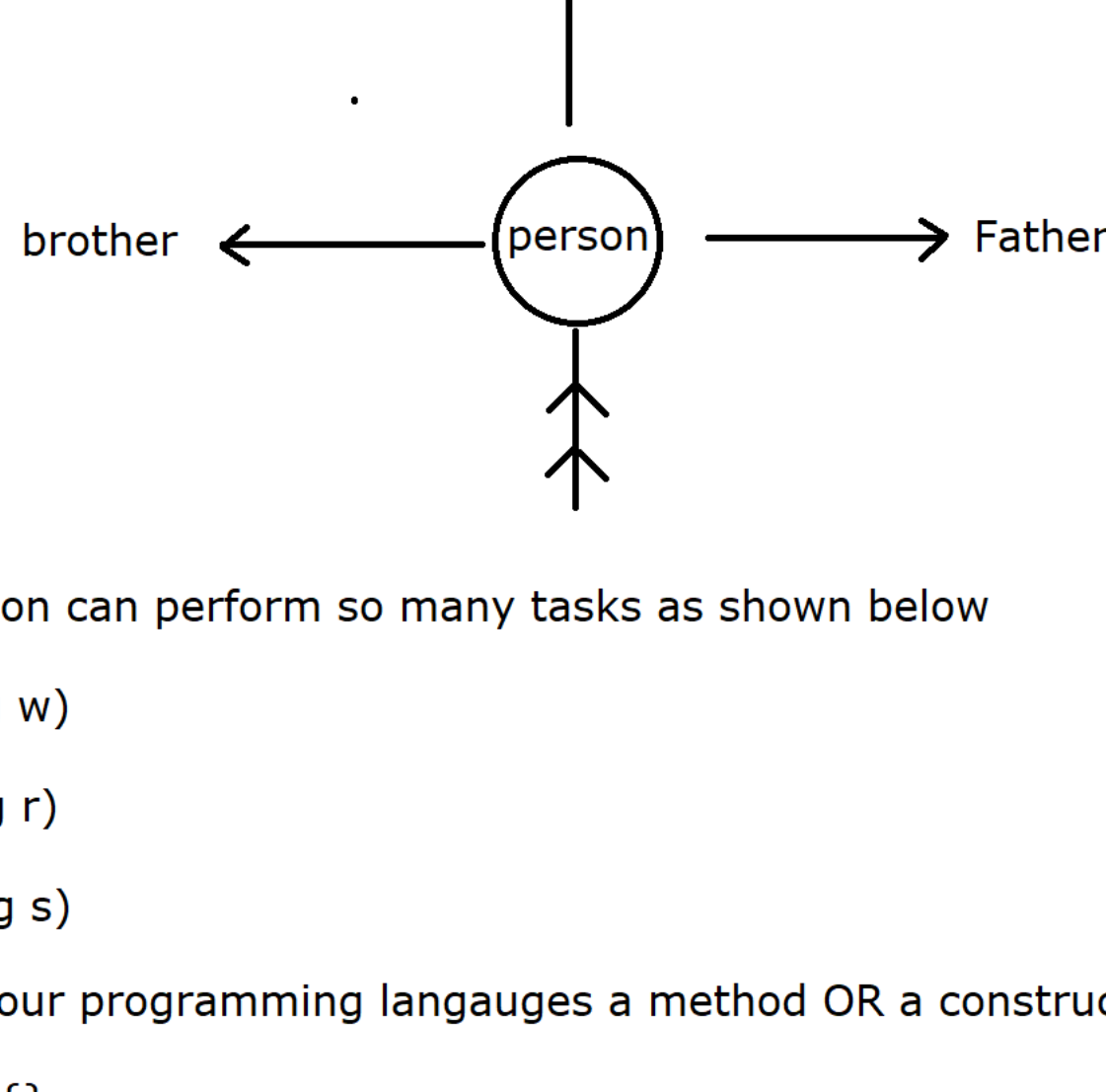
```
public class CoVariantDemo1
{
    public static void main(String[] args)
    {
        Shape s = new Square();
        s.draw();
    }
}
```

```
class Vehicle
{
    public Object run()
    {
        IO.println("Generic Run");
        return null;
    }
}
class Car extends Vehicle
{
    public Double run()
    {
        IO.println("Car is Running");
        return null;
    }
}
public class CoVariantDemo2
{
    public static void main(String[] args)
    {
        Vehicle v = new Car();
        v.run();
    }
}
```

Polymorphism :

* Poly means "many" and morphism means "Forms"

* Polymorphism is a Greek word whose meaning "Object can have different behaviour".



* In our real life a person can perform so many tasks as shown below

```
void person(Walking w)
```

```
void person(Running r)
```

```
void person(Sleeping s)
```

* In the same way, In our programming languages a method OR a constructor can perform so many task

```
void add(int x, int y){}
```

```
void add(int x, int y, int z){}
```

```
void add(double x, double y){}
```

* Polymorphism is divided into two types :



Compile time Polymorphism

OR

Static Polymorphism

OR

Early Binding

Runtime Polymorphism

OR

Dynamic Polymorphism

OR

Late Binding

* Final Keyword in java [sealed keyword]

* Object class methods

* Abstract class and abstract methods

* Inner class

* Interface JDK 1.8 V

Arrays & String Logical Programs (pattern programming)

Exception Handling

Multithreading (IO & File Handling)

Collections (Generics + Stream API)